

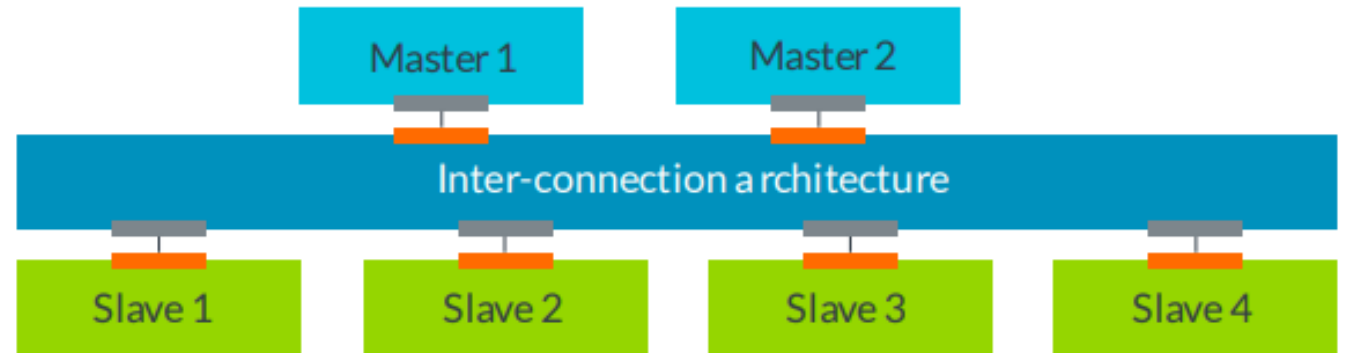
AXI Bus Communication Protocol Verification

Project Overview

- Design and verify the AMBA AXI bus protocol using SystemVerilog.
- Ensure proper data transfer, address decoding, and multi-master/slave handling.
- Validate the design through simulation and functional coverage analysis.
- Implement and verify burst transactions, multiple outstanding transactions, and error handling.



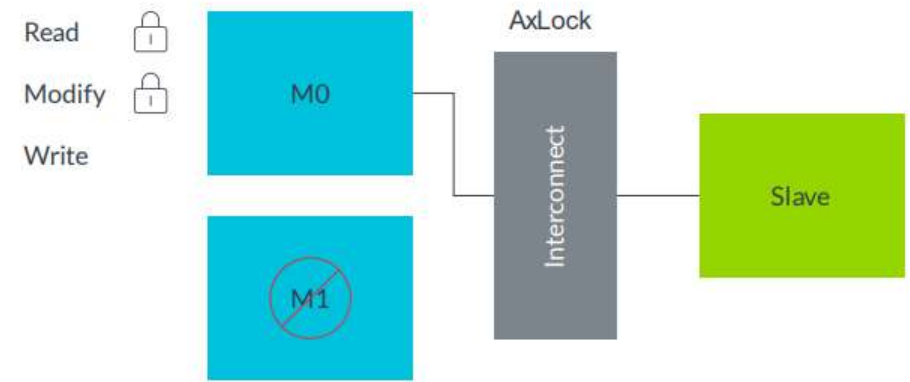
Design Components



- **AXI Master** – Initiates read/write transactions.
- **AXI Slave** – Responds to requests and provides data.
- **AXI Interconnect** – Routes transactions to the correct slave based on address decoding.
- **AXI Bus** – Transfers data, address, and control signals.

AXI Interconnect Module

- Routes transactions between the AXI Master and multiple AXI Slaves.
- Performs address decoding to direct requests to the correct slave.
- Handles multiple outstanding transactions.
- Ensures proper response handling for read/write operations.



Verification Plan

- Directed test cases for read/write operations.
- Burst transactions verification.
- Multiple outstanding transactions and out-of-order execution.
- Error handling (invalid addresses, slave readiness failures).
- Functional coverage analysis to ensure all features are tested.

Testbench Overview

```
module axi_tb;

// Declare signals
reg clk;
reg reset;
reg [31:0] addr;
reg [31:0] wr_data;
wire [31:0] rd_data;
reg [3:0] axi_arvalid;
reg [3:0] axi_arinvalid;
reg [3:0] axi_wvalid;
wire [3:0] axi_rvalid;
wire [3:0] axi_bvalid;
reg [3:0] axi_rready;
reg [3:0] axi_bready;

// Instantiate master, slave, and interconnect
axi_master master1 (
    .clk(clk),
    .reset(reset),
    .addr(addr),
    .wr_data(wr_data),
    .axi_arvalid(axi_arvalid),
    .axi_arinvalid(axi_arinvalid),
    .axi_wvalid(axi_wvalid),
    .axi_rvalid(axi_rvalid),
    .axi_bvalid(axi_bvalid),
    .axi_rready(axi_rready),
    .axi_bready(axi_bready)
);

axi_slave slave1 (
    .clk(clk),
    .reset(reset),
    .addr(addr),
    .wr_data(wr_data),
    .rd_data(rd_data),
    .axi_arvalid(axi_arvalid),
    .axi_arinvalid(axi_arinvalid),
    .axi_wvalid(axi_wvalid),
    .axi_rvalid(axi_rvalid),
    .axi_bvalid(axi_bvalid)
);

// Clock generation
always #5 clk = ~clk;

// Reset logic
initial begin
    clk = 0;
    reset = 1;
    #10 reset = 0;
end

// Test sequence
initial begin
    // Test 1: Single Read Transaction
    addr = 32'h0000_0000;
    wr_data = 32'hA5A5_A5A5;
    axi_arvalid = 4'b0001;
    axi_wvalid = 4'b0000;
    #10;
    axi_arinvalid = 4'b0000;
    #10; // Wait for response

    // Test 2: Burst Write Transaction
    addr = 32'h1000_0000;
    wr_data = 32'h1234_5678;
    axi_arvalid = 4'b0001;
    axi_wvalid = 4'b0001;
    #20;
    axi_arinvalid = 4'b0000;
    axi_wvalid = 4'b0000;
    #10;

    // Test 3: Out-of-Order Transactions
    addr = 32'h2000_0000;
    wr_data = 32'hDEAD_BEEF;
    axi_arvalid = 4'b0001;
    axi_wvalid = 4'b0001;
    #10;
    addr = 32'h3000_0000;
    wr_data = 32'hBEEF_DEAD;
    axi_arvalid = 4'b0001;
    axi_wvalid = 4'b0001;
    #10;
    axi_arinvalid = 4'b0000;
    axi_wvalid = 4'b0000;
    #10;

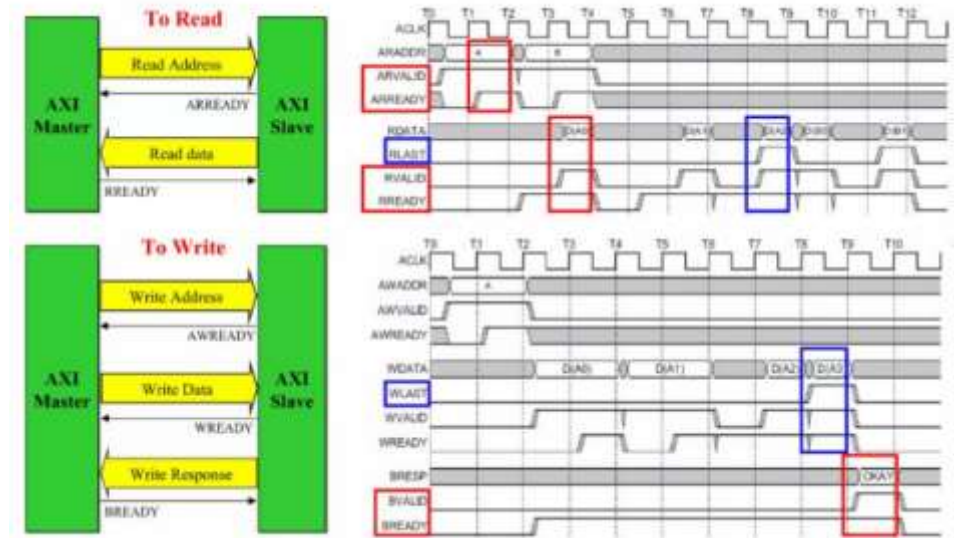
    // Test 4: Error Handling
    addr = 32'hFFF0_FFFF;
    wr_data = 32'h9999_9999;
    axi_arvalid = 4'b0001;
    axi_wvalid = 4'b0001;
    #10;
    axi_arinvalid = 4'b0000;
    axi_wvalid = 4'b0000;
    #10;

    end
endmodule
```

- Instantiates AXI Master, AXI Interconnect, and AXI Slaves.
- Generates test scenarios to validate transactions.
- Checks read/write transactions, burst handling, and error conditions.
- Monitors system responses using assertions and waveform analysis.

Simulation and Debugging

- Waveform analysis in ModelSim to verify AXI transactions.
- Address decoding validation.
- Burst and multiple outstanding transaction handling.
- Error handling for invalid memory accesses.
- Functional coverage tracking for complete verification.



Conclusion

- **Successfully designed and verified an AXI bus communication system.**
- **Ensured correct data transfer, address decoding, and multi-slave support.**
- **Implemented and validated burst transactions and error handling.**
- **Simulated and debugged the system using ModelSim.**

Thank you!